

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	M.KASIF JALAL
Name	192424214

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I M.KASIF JALAL certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: KASIF

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. **Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.**

ANS:

- A media platform generates a very large amount of data in the form of videos, images, audio files, subtitles, user activity logs, metadata, and application records. Since media files are generally large and continuously generated, a distributed storage system such as the Hadoop Distributed File System (HDFS) is suitable. HDFS is designed to store large files across multiple commodity servers while providing fault tolerance and high-throughput data access.
- The major constraints in this design are that data availability must remain above 99.9%, the replication factor must not exceed 3, and storage growth must remain cost efficient. Therefore, the proposed architecture uses a three-way HDFS replication strategy, rack-aware placement, efficient file organization, compression, archival policies, and horizontal scaling.

Proposed HDFS Architecture

The HDFS cluster consists of a NameNode, DataNodes, and supporting services. The NameNode maintains metadata such as file names, block locations, permissions, and replication information. DataNodes store the actual data blocks.

For high availability, two NameNodes can be used:

- Active NameNode
- Standby NameNode

The Standby NameNode continuously maintains a synchronized copy of metadata and can take over if the Active NameNode fails. ZooKeeper and automatic failover mechanisms can be used to coordinate NameNode availability.

The DataNode layer contains multiple commodity servers distributed across different racks. Large media files are divided into HDFS blocks, for example 128 MB or 256 MB blocks, and these blocks are distributed across DataNodes.

Replication Strategy

The replication factor is limited to a maximum of three. Therefore, a replication factor of **3** is selected for important media data.

For example, if a video is divided into blocks B1, B2, and B3, HDFS can store:

- One copy on DataNode 1
- Second copy on DataNode 2
- Third copy on DataNode 5

The replicas should be distributed across different racks whenever possible. This is called **rack-aware replication**.

Rack-aware placement is important because a failure may affect an entire rack rather than a single server. If all three replicas were stored in the same rack, a rack-level power or network failure could make the data unavailable. By placing replicas across different racks, the probability of simultaneous loss is greatly reduced.

Achieving More Than 99.9% Availability

Availability greater than 99.9% requires eliminating single points of failure.

The following mechanisms are used:

1. **NameNode High Availability:** An Active and Standby NameNode are maintained.
2. **Data Replication:** Important blocks have three copies.
3. **Rack Awareness:** Replicas are distributed across multiple racks.
4. **Automatic Failover:** If the Active NameNode fails, the Standby NameNode can become active.
5. **DataNode Heartbeats:** NameNode continuously monitors DataNodes.
6. **Automatic Re-replication:** If a DataNode fails, HDFS creates another replica on a healthy DataNode.
7. **Checksums:** HDFS verifies data integrity and detects corrupted blocks.

These mechanisms ensure that a hardware failure does not immediately result in data unavailability.

Cost-Efficient Storage Growth

Although replication improves reliability, storing three copies of every file increases storage requirements. Therefore, cost efficiency is important.

The platform can divide data into different storage categories:

Hot data: Recently uploaded or frequently accessed videos are stored on high-performance storage.

Warm data: Less frequently accessed content can be moved to relatively inexpensive storage.

Cold/archive data: Old videos and historical logs can be moved to lower-cost archival storage.

Compression can also reduce storage requirements. Text-based metadata, logs, subtitles, and other compressible files can use formats such as Snappy, Gzip, or similar Hadoop-supported compression techniques.

The organization should also avoid storing unnecessary duplicate copies at the application level. HDFS replication should be responsible for fault tolerance rather than creating additional application-level copies.

File and Block Management

Large media files should be stored as large HDFS files rather than thousands of very small files. The small-file problem can create excessive metadata overhead on the NameNode.

Media metadata can be maintained separately from large media objects. For example:

- /media/videos/
- /media/images/
- /media/audio/
- /media/metadata/
- /media/logs/
- /media/archive/

Partitioning data according to content type, date, and region can improve management and analytics.

For example:

/media/videos/year=2026/month=08/day=24/

This allows applications to access only the required partitions instead of scanning the complete dataset.

Scalability

HDFS provides horizontal scalability. When storage requirements increase, additional DataNodes can be added to the cluster. The existing data can then be balanced across the available nodes.

For example, if the platform initially has 20 DataNodes and storage requirements double, additional DataNodes can be added without replacing the existing servers.

This makes HDFS suitable for media platforms because data volume can grow continuously.

Overall Design

The proposed architecture can be summarized as:

Media Sources → Ingestion Layer → HDFS → Data Processing → Metadata/Analytics → Applications

Within HDFS:

Active NameNode + Standby NameNode → Multiple Rack-Aware DataNodes → 3-Way Replication

The design provides fault tolerance while maintaining the replication factor within the specified limit.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

ANS:

- Large applications continuously generate log data containing user activities, server requests, errors, authentication events, transactions, and system performance information. When the processing infrastructure has limited cluster nodes, the MapReduce workflow must be carefully designed to complete within the required 20-minute batch window.
- The main objective is to divide the log-processing workload into parallel tasks while minimizing unnecessary data movement and reducing the amount of intermediate data transferred between Map and Reduce stages.

Proposed MapReduce Workflow

The proposed workflow consists of the following stages:

Log Files → Input Splits → Mapper → Combiner → Shuffle and Sort → Reducer → Output

The log data is first stored in HDFS. The input files are divided into blocks and corresponding input splits. Multiple Mapper tasks process these splits in parallel.

For example, suppose the cluster has six worker nodes and the log data is divided into 24 input splits. Multiple mapper tasks can be distributed among the six nodes.

Mapper Design

The Mapper reads each log record and extracts important fields.

A typical log record may contain:

- Timestamp
- IP address
- User ID
- URL
- HTTP status code
- Response time
- Device information

The Mapper can convert each record into a key-value pair.

For example:

HTTP 500 error → (500, 1)

or:

Page /home accessed → (/home, 1)

For user activity analysis:

User123 → (page_view, 1)

The Mapper should perform filtering as early as possible. Invalid records, unnecessary fields, and irrelevant events should be removed during the Map stage.

Use of Combiner

A Combiner should be used whenever the aggregation operation permits it.

For example, if one Mapper processes 10,000 occurrences of HTTP 500 errors, instead of sending 10,000 intermediate records to the Reducer, the Combiner can produce:

500 → 10,000

This significantly reduces network traffic during the Shuffle phase.

Using a Combiner is especially important when the cluster has limited nodes because network bandwidth and CPU resources are restricted.

Reducer Design

The Reducer receives grouped intermediate values.

For example:

500 → [1,1,1,1,1,...]

The Reducer calculates:

500 → total_count

The final output may contain:

- Number of requests
- Number of errors
- Number of successful requests
- Average response time
- Number of active users
- Most frequently accessed pages

The number of Reducers should be selected according to available CPU cores, expected intermediate data volume, and the number of cluster nodes.

Task Distribution

Because the cluster contains limited nodes, task distribution must maximize parallelism without causing excessive resource contention.

Suppose there are six worker nodes:

- Node 1: Mapper tasks
- Node 2: Mapper tasks
- Node 3: Mapper + Reducer
- Node 4: Mapper + Reducer
- Node 5: Reducer
- Node 6: Reducer

The actual distribution is controlled by YARN, which allocates containers based on available CPU and memory. Map tasks should preferably execute on nodes where the corresponding HDFS blocks are stored. This is called **data locality**.

Data locality reduces network transfer because the Mapper processes data close to where it is stored.

Meeting the 20-Minute Constraint

To meet the 20-minute deadline, several optimizations should be applied.

1. Increase Map Parallelism

Large input data should be divided into multiple input splits so that several Mapper tasks can work simultaneously.

2. Use Data Locality

Map tasks should run on DataNodes containing the required HDFS blocks whenever possible.

3. Use a Combiner

Combining intermediate values before Shuffle reduces network communication.

4. Avoid Unnecessary Processing

The Mapper should extract only the fields required for the final analysis.

5. Compress Intermediate Data

Intermediate Map output can be compressed before being transferred to Reducers.

6. Select Appropriate Reducer Count

Too few Reducers create bottlenecks, while too many Reducers create scheduling and coordination overhead. A balanced number should be selected based on the available nodes.

Example Task Allocation

Assume there are 12 input splits and four worker nodes.

The initial distribution could be:

- Node 1 → Splits 1–3
- Node 2 → Splits 4–6
- Node 3 → Splits 7–9
- Node 4 → Splits 10–12

Each node executes Mapper tasks in parallel.

After mapping and combining, the Shuffle phase distributes records according to their keys. For example, all HTTP 404 records are sent to the Reducer responsible for that partition.

The Reducers then generate the final output in HDFS.

Handling Slow or Failed Tasks

A limited-node environment cannot afford long delays caused by failed tasks. MapReduce provides fault tolerance by allowing failed tasks to be restarted.

If one task becomes significantly slower than others, speculative execution may launch another copy of the task on another available container. The result from the task that completes successfully first is used.

However, speculative execution should be controlled because excessive duplicate execution could consume limited cluster resources.

Job Configuration

The workflow should use:

- HDFS for input and output

- YARN for resource management
- MapReduce for batch processing
- Multiple Mapper tasks for parallel processing
- Combiner for local aggregation
- Appropriate number of Reducers
- Data-local execution
- Intermediate compression
- Controlled speculative execution
- The proposed MapReduce workflow is designed specifically for a limited-node environment with a 20-minute processing requirement. Parallel input splits increase Map-stage concurrency, data locality minimizes network traffic, Combiners reduce intermediate data, and an appropriately selected number of Reducers prevents bottlenecks. YARN dynamically distributes tasks according to available resources, while MapReduce provides automatic recovery from failed tasks. With these optimizations, the log-processing job can be designed to complete reliably within the 20-minute batch window.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

ANS:

- YARN is the resource management layer of the Hadoop ecosystem. It allows multiple applications to share a Hadoop cluster while controlling CPU, memory, and other resources. In an enterprise environment, the same cluster may run ETL jobs, reporting workloads, and machine learning applications.
- These workloads have different requirements. ETL jobs may be time-sensitive because they prepare data for downstream applications. Reporting requires predictable access to resources, while machine learning jobs may consume large amounts of CPU and memory for longer periods.
- Therefore, a scheduler policy must provide fairness while giving ETL jobs higher priority and ensuring that workloads recover after node failures.

Proposed Scheduler

The recommended approach is to use the **Capacity Scheduler** with separate queues.

The cluster can be divided into three major queues:

- ETL Queue
- Reporting Queue
- Machine Learning Queue

A possible allocation is:

- ETL: 50%
- Reporting: 25%
- Machine Learning: 25%

These values represent guaranteed capacity rather than permanent resource ownership. If one queue is not using its allocated resources, other queues can temporarily use the unused capacity.

ETL Priority

ETL is given the highest priority because ETL processes transform and prepare raw data for other applications.

Within the ETL queue, jobs can be assigned different priorities.

For example:

Priority 1: Critical daily ETL

Priority 2: Important scheduled ETL

Priority 3: Non-critical historical ETL

The scheduler should allocate resources to critical ETL workloads before lower-priority ETL jobs.

This prevents an important data pipeline from being delayed by a large machine learning experiment.

Fairness

Although ETL receives priority, reporting and machine learning jobs should not be starved.

Fairness can be achieved through queue capacity and resource limits.

For example, the Reporting queue receives guaranteed capacity even when ETL activity is high. Similarly, the Machine Learning queue receives a guaranteed minimum allocation.

The scheduler can allow unused capacity to be temporarily borrowed.

For example, if the Reporting queue is using only 10% of the cluster while ETL requires additional resources, ETL can temporarily use the unused reporting capacity.

When reporting demand increases, those resources can be returned through scheduling mechanisms.

User and Application Limits

Resource limits should also be defined for individual users and applications.

For example:

- One user cannot consume the entire ML queue.
- One reporting application cannot consume all reporting resources.
- A single ETL application cannot exceed a defined maximum capacity unless explicitly permitted.

This prevents resource monopolization.

Machine Learning Workloads

Machine learning jobs often run for long periods and may require significant memory and CPU. They should therefore be placed in a separate queue.

The ML queue can support:

- Long-running training jobs
- Model evaluation
- Feature engineering
- Hyperparameter experiments

The queue should have a maximum resource limit so that ML experiments do not negatively affect critical ETL and reporting workloads.

Node Failure Recovery

Node failure is an important requirement in a distributed Hadoop cluster.

YARN's ResourceManager monitors available cluster resources, while NodeManagers run on individual worker nodes.

If a NodeManager stops responding, the ResourceManager detects the failure through missed heartbeats.

The failed node is marked unavailable, and resources are no longer allocated to it.

Applications running tasks on that node can recover by restarting the affected tasks on healthy nodes.

For example:

Node 4 fails → ResourceManager detects failure → Node 4 removed from active resource pool → Failed containers/tasks restarted → Tasks complete on healthy nodes

For long-running applications, application-level checkpointing can further reduce recovery time.

Scheduler Policy

The complete policy can be summarized as follows:

Root Queue

→ ETL Queue — 50%

→ Reporting Queue — 25%

→ Machine Learning Queue — 25%

Within ETL:

→ Critical ETL

→ Normal ETL

→ Background ETL

Within Reporting:

→ Scheduled reports

→ Interactive reports

Within ML:

→ Production training

→ Research experiments

Production ML jobs may receive greater priority than experimental workloads.

Handling Resource Contention

Suppose an ETL job suddenly requires 60% of the cluster. The scheduler gives the ETL queue its guaranteed capacity first and allows it to borrow unused capacity from other queues.

However, reporting and ML workloads retain minimum guaranteed resources.

This provides a balance between priority and fairness.

Monitoring and Recovery

The cluster should continuously monitor:

- CPU utilization
- Memory utilization
- Queue utilization
- Application completion time
- Failed containers
- Node health
- Job waiting time

If repeated failures occur on a particular node, the node can be temporarily excluded from scheduling until it becomes healthy.

- The proposed YARN scheduling policy uses separate queues for ETL, reporting, and machine learning. ETL receives the highest priority and guaranteed capacity, while reporting and ML workloads receive fair minimum allocations. Unused resources can be temporarily shared to improve overall cluster utilization. Node failures are detected through YARN heartbeat mechanisms, and affected tasks are restarted on healthy nodes. Therefore, the policy provides a practical balance between fairness, ETL priority, resource utilization, and fault recovery.

4. **An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.**

ANS:

- An enterprise may store information across different systems such as Storage Area Networks (SAN), Network Attached Storage (NAS), and operational relational databases. SAN may contain high-performance enterprise data, NAS may contain documents and files, and operational databases may contain structured transaction records.
- The challenge is to bring these heterogeneous data sources into a centralized big data platform while maintaining reliable backups and avoiding unnecessary duplication.
- The proposed architecture uses an ingestion layer, centralized Hadoop storage, metadata management, incremental database extraction, controlled backup, and deduplication mechanisms.

Proposed Architecture

The overall architecture is:

SAN + NAS + Operational Databases

↓

Apache NiFi / ETL Ingestion Layer

↓

Validation + Deduplication + Metadata

↓

HDFS / Data Lake

↓

Processing and Analytics

↓

Reporting / Machine Learning / Applications

A backup system is integrated with the storage layer.

SAN Integration

SAN generally provides block-level storage to enterprise servers. Data stored in SAN should not simply be copied repeatedly into HDFS.

An ingestion agent or ETL process can access authorized SAN volumes and transfer required datasets into the Hadoop environment.

Only datasets required for analytics should be ingested.

Before ingestion, the system should calculate checksums or unique identifiers. This helps determine whether the same file has already been transferred.

NAS Integration

NAS provides file-level access and may contain:

- Documents

- Images
- Videos
- CSV files
- XML files
- JSON files
- Reports

Apache NiFi can monitor NAS directories and automatically ingest newly created or modified files.

NiFi processors can perform:

- File listing
- File fetching
- Validation
- Routing
- Transformation
- Compression
- Metadata extraction

After successful ingestion, the file can be marked as processed to prevent repeated transfers.

Operational Database Integration

Operational databases contain structured records such as:

- Customer information
- Orders
- Payments
- Inventory
- Employee records

Instead of copying the entire database repeatedly, incremental extraction should be used.

For example, a column such as:

`last_updated_timestamp`

can identify records modified since the previous ingestion.

Alternatively, Change Data Capture (CDC) can capture INSERT, UPDATE, and DELETE operations.

This reduces network traffic, processing requirements, and duplication.

Apache NiFi Ingestion Layer

Apache NiFi can serve as the central ingestion and data-flow management layer.

Separate flows can be designed for each source:

SAN → NiFi → HDFS

NAS → NiFi → HDFS

Database → NiFi → HDFS

NiFi provides flow management, routing, monitoring, back-pressure, and provenance tracking.

Back-pressure is particularly useful because it prevents an overloaded downstream system from receiving unlimited data.

Minimal Duplication

Duplication can occur when the same file or database record is transferred multiple times.

The following mechanisms can reduce duplication:

1. File checksums
2. Unique file identifiers
3. Database primary keys
4. Incremental extraction
5. Change Data Capture
6. Metadata tracking
7. Idempotent ingestion processes

For files, a hash can be calculated. If the same hash already exists in the data lake, the system can avoid storing another copy.

For database records, primary keys and timestamps can be used to determine whether a record is new or modified.

HDFS Storage Organization

The data lake should be divided into logical zones:

Raw Zone: Original data is preserved.

Processed Zone: Cleaned and transformed data is stored.

Curated Zone: Business-ready datasets are stored.

For example:

/data/raw/san/

/data/raw/nas/

/data/raw/database/

/data/processed/

/data/curated/

This organization makes data management easier.

Backup Reliability

Backup reliability should be achieved without creating unnecessary copies.

HDFS replication can provide protection against individual DataNode failures. For critical data, a replication factor of three can be used.

However, replication alone should not be considered a complete backup strategy because all replicas may exist within the same infrastructure.

Important data should therefore also be backed up to an independent storage system or secondary cluster.

Backup policies can use:

- Periodic snapshots
- Secondary storage

- Off-site backup
- Incremental backup
- Disaster recovery copies

Only critical datasets need long-term retention to control storage costs.

Metadata Management

A metadata catalog should record:

- Source system
- File name
- Record type
- Ingestion time
- Owner
- Schema
- Data quality status
- Checksum
- Processing status

This prevents confusion and helps determine whether data has already been ingested.

Security

Data movement should use secure protocols and authenticated connections. Access to SAN, NAS, databases, and HDFS should be controlled using role-based permissions.

Sensitive operational data should also be encrypted both during transmission and at rest.

- The proposed architecture uses Apache NiFi or an equivalent ETL layer to connect SAN, NAS, and operational databases to a centralized Hadoop data lake. Incremental extraction and CDC minimize unnecessary database duplication, while checksums and metadata tracking prevent repeated file ingestion. HDFS replication provides storage fault tolerance, while independent backups provide disaster recovery. Logical raw, processed, and curated zones simplify data management. Therefore, the architecture provides reliable ingestion, minimal duplication, scalable storage, and dependable backup protection.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

ANS:

- An enterprise Hadoop platform must collect data from multiple sources, securely move that data into the Hadoop ecosystem, process it at scale, and make the results available to applications and analytics systems. The architecture must also remain available even when individual servers or services fail.

The proposed solution integrates Apache NiFi with HDFS, YARN, Hive, Spark, security services, and monitoring components. The architecture focuses on three major constraints:

1. Secure data movement
2. High availability
3. Integration with Apache NiFi or ETL pipelines

Overall Architecture

The proposed architecture is:

Data Sources

→ Operational Databases

→ Files

→ Applications

→ IoT / Logs

↓

Apache NiFi / ETL Layer

↓

Secure Data Transfer

↓

HDFS Data Lake

↓

YARN Resource Management

↓

Spark / MapReduce Processing

↓

Hive / Data Warehouse Layer

↓

BI Dashboards / Machine Learning / Applications

Security and monitoring operate across all layers.

Data Ingestion Using Apache NiFi

Apache NiFi is used as the primary ingestion and data-flow management component.

NiFi can collect data from:

- Relational databases
- Files
- APIs
- Log systems
- Network sources
- Enterprise applications

NiFi processors can perform extraction, transformation, filtering, routing, and loading.

For example:

Database → NiFi → Validation → Transformation → HDFS

For files:

File System → NiFi → Duplicate Check → Metadata → HDFS

This allows the enterprise to manage different ingestion pipelines from a centralized interface.

Secure Data Movement

Security must be maintained while data moves between sources and Hadoop.

Encrypted protocols such as TLS/HTTPS should be used for communication between systems where supported.

For database ingestion, NiFi should connect using authenticated and encrypted database connections.

Access credentials should not be embedded directly in application code. They should be managed securely through appropriate credential-management mechanisms.

Hadoop services should also use authentication and authorization mechanisms. Kerberos can be used to authenticate Hadoop users and services in enterprise environments.

Data at rest can be protected using encryption mechanisms supported by the Hadoop platform.

HDFS Storage Layer

HDFS provides distributed storage for the incoming data.

The data can be divided into three logical zones:

Raw Zone: Stores data in its original form.

Processed Zone: Stores cleaned and transformed data.

Curated Zone: Stores analytics-ready data.

For example:

/data/raw/

/data/processed/

/data/curated/

This separation allows the enterprise to preserve original data while providing clean datasets for analytics.

High Availability

High availability is required at the critical Hadoop service level.

The NameNode should use an Active/Standby architecture.

The Active NameNode manages HDFS metadata while the Standby NameNode maintains synchronized metadata and can take over during failure.

DataNodes provide distributed storage and replication.

With a replication factor of three, important HDFS blocks can have three copies distributed across different DataNodes and racks.

If one DataNode fails, another replica can serve the data.

YARN Resource Management

YARN manages computing resources for applications.

The cluster can execute:

- Spark jobs
- MapReduce jobs
- Hive queries
- Machine learning applications
- ETL processing

YARN allocates CPU and memory through containers.

Different workloads can be separated into queues. For example:

ETL Queue

Analytics Queue

Machine Learning Queue

This prevents one workload from consuming all cluster resources.

Data Processing

Apache Spark can be used for large-scale processing because it supports efficient batch and iterative analytics.

MapReduce can also be used for traditional batch-processing workloads.

Hive provides SQL-based access to data stored in Hadoop.

For example, business users can query curated sales data using Hive SQL without directly writing MapReduce programs.

ETL Pipeline Integration

NiFi can perform initial ingestion and lightweight transformation. For complex transformations, NiFi can pass the data to Spark or other ETL processing engines.

A typical pipeline is:

Source → NiFi → Validation → HDFS Raw → Spark ETL → HDFS Processed → Hive → BI

This separation prevents the ingestion layer from becoming overloaded with complex processing.

Reliability and Failure Recovery

The architecture should monitor all major services.

NiFi provides flow monitoring and provenance information. Hadoop monitors DataNodes and NameNodes. YARN monitors worker nodes and application containers.

If a worker node fails:

Failure detected → Resources removed → Tasks restarted → Processing continues

If a DataNode fails, HDFS serves data from another replica and creates a replacement replica when necessary.

If the Active NameNode fails, the Standby NameNode can take over through automatic failover mechanisms.

Data Quality

Before data enters the main processing layer, NiFi can validate:

- Required fields
- Data types
- File formats
- Record structure
- Duplicate records
- Invalid values

Invalid data can be routed to a quarantine area instead of contaminating the main dataset.

This improves the quality of downstream analytics.

Monitoring

The platform should continuously monitor:

- CPU usage
- Memory usage

- Disk capacity
- DataNode health
- NameNode health
- YARN applications
- NiFi queues
- Failed ingestion flows
- Processing time

Alerts should be generated when storage capacity, processing time, or failure rates exceed predefined thresholds.

How the Architecture Satisfies the Constraints

Secure Data Movement: TLS/HTTPS, authentication, authorization, Kerberos, controlled credentials, and encryption protect data during movement and storage.

High Availability: Active/Standby NameNodes, HDFS replication, rack-aware placement, automatic failover, and task recovery reduce downtime.

NiFi/ETL Integration: Apache NiFi provides centralized ingestion, routing, validation, transformation, and flow management. Spark and Hive can perform advanced ETL and analytics.

Scalability: Additional DataNodes and worker nodes can be added when data volume or processing requirements increase.

Reliability: Replication, monitoring, automatic task recovery, and backup mechanisms protect against

- The proposed end-to-end Hadoop ecosystem combines Apache NiFi, HDFS, YARN, Spark, MapReduce, and Hive into a secure and highly available data platform. NiFi provides flexible data ingestion and ETL integration, while HDFS provides scalable distributed storage. YARN manages computing resources, Spark and MapReduce perform large-scale processing, and Hive supports SQL-based analytics. Authentication, encryption, authorization, replication, high-availability NameNodes, monitoring, and automatic recovery satisfy the security and availability requirements. Therefore, the architecture provides a complete, scalable, secure, and reliable Hadoop solution for enterprise big data processing.